ATIVIDADE · APIs, AUTENTICAÇÃO E CACHE LOCAL

ReUse Mobile

Integração com APIs, sessões persistentes e experiência offline

Aluna	**Tamyres Ogasawara Macedo**
RM	**552055**
Modalidade	**Trabalho individual**
Instituição	**FIAP**

Repositório GitHub: github.com/tamyresogasawara/reuse-mobile-fiap-sprint-2

Aplicação web: tamyresogasawara.github.io/reuse-mobile-fiap-sprint-2

Outubro de 2026

1. Visão geral da solução

Como a implementação atende aos três critérios da atividade

A ReUse da Sprint 2 — com busca, favoritos, anúncios por câmera e gamificação — passou a ser conectada à API pública DummyJSON. O fluxo agora vai das boas-vindas ao login, restaura a sessão ao reabrir o aplicativo e oferece um catálogo online com cache local e fallback offline, sem perder nenhuma funcionalidade anterior.

Critério	Peso	Entrega implementada
Consumo de API	60%	Login por POST e catálogo por GET, com validação da resposta, carregamento, atualização manual, retry e mensagens de erro.
Autenticação e sessões	20%	Tokens e perfil persistidos; restauração automática; nome e usuário na Início e no Perfil; logout.
Cache local	20%	Catálogo salvo por 15 minutos; cache recente sem rede e último cache como fallback offline.

Arquitetura principal

- SessionProvider (src/state/SessionContext.tsx): restaura e valida a sessão antes de montar as rotas e expõe login e logout.
- App.tsx: sem sessão, mostra Boas-vindas e Login; com sessão, libera as abas, detalhes, catálogo online, Meus anúncios e Meu impacto.
- src/services/auth.ts: autenticação, normalização, persistência, restauração, expiração e logout.
- src/services/products.ts: consumo do catálogo, validação, TTL e fallback do cache.
- ExploreScreen (Catálogo online): estados de rede, cache, offline, erro, retry e atualização manual.
- Testes automatizados: serviços, telas, navegação, sessão, gamificação e movimento.

2. APIs utilizadas

Duas integrações externas cobrem autenticação e conteúdo

API PÚBLICA · DUMMYJSON

2.1 Autenticação

Endpoint: POST https://dummyjson.com/auth/login

A LoginScreen valida os campos e envia username, password e validade de 60 minutos. A resposta é validada antes de gerar uma sessão com accessToken, refreshToken e dados normalizados do usuário. Credenciais recusadas (HTTP 400) e outras falhas produzem mensagens claras sem liberar a navegação.

- Telas afetadas: Boas-vindas → LoginScreen → navegação principal.
- Dados utilizados: accessToken, refreshToken, id, username, firstName e lastName.
- Conta pública de avaliação: emilys / emilyspass.

2.2 Catálogo de produtos

Endpoint: GET https://dummyjson.com/products?
limit=12&select=id,title,description,price,category,thumbnail

O cartão Catálogo online da Início abre a ExploreScreen, que apresenta doze produtos recebidos da API com miniatura, título, categoria, descrição e preço formatado em reais. A tela mostra a origem dos dados: rede, cache recente ou cache expirado em modo offline.

- Tela afetada: ExploreScreen (Catálogo online).
- Estados visuais: carregando, sucesso, erro, retry, cache e offline.
- Atualização manual: o botão Atualizar catálogo ignora o TTL e força nova requisição.

3. Autenticação e gerenciamento de sessão

Fluxo protegido do acesso ao logout



Figura 1 — Login integrado à API DummyJSON.

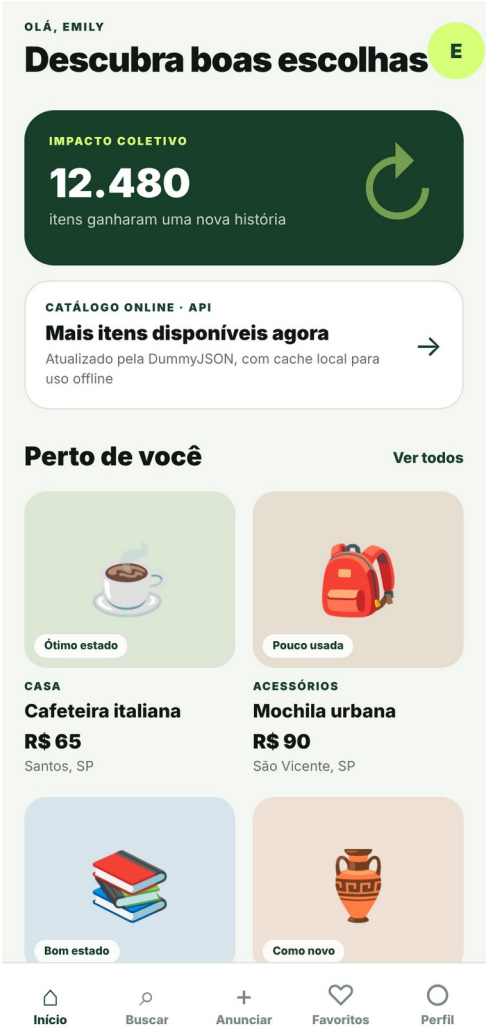


Figura 2 — Início autenticada com o nome da sessão e o catálogo online.

No Android e iOS, os tokens são persistidos com Expo SecureStore. Na web, o aplicativo usa Async Storage, porque o SecureStore não existe no navegador. Ao iniciar, o SessionProvider restaura e valida a sessão salva antes de montar as rotas protegidas.

- Sem sessão válida ou após 60 minutos: a pessoa vê Boas-vindas e Login.
- Sessão válida: o aplicativo abre direto na navegação principal.
- Logout: o botão Sair do Perfil remove a sessão e retorna às boas-vindas.
- Dados locais inválidos: são descartados em vez de quebrar o aplicativo.



Figura 3 — Perfil com o usuário autenticado e a opção Sair.

O Perfil identifica a pessoa pelo nome e usuário recebidos da API e explica onde os tokens ficam guardados. Favoritos, rascunhos, anúncios e pontos de impacto continuam somente no dispositivo.

4. Cache local e experiência offline

Catálogo rápido e resiliente a falhas de rede

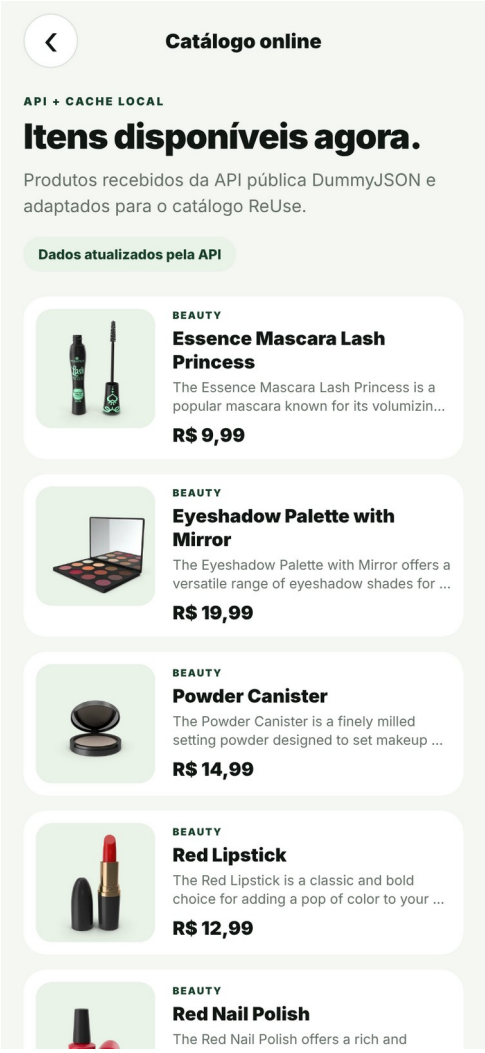


Figura 4 — Catálogo com dados atualizados pela API.

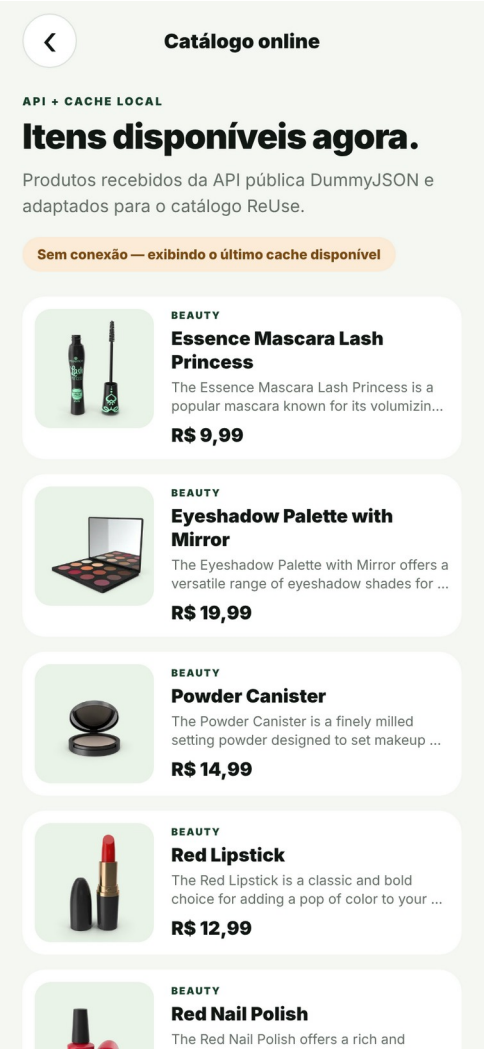


Figura 5 — Rede indisponível (simulada): último cache com aviso.

O cache usa a chave @reuse/products-v1 e guarda a lista validada de produtos com savedAt. O prazo de validade é de 15 minutos.

Regras de leitura

- Cache recente: retorna imediatamente sem chamada à API.
- Cache ausente ou expirado: consulta a rede e salva uma nova versão.
- Falha de rede com cache disponível: apresenta o último catálogo com aviso offline.
- Falha sem cache: mostra erro e botão para tentar novamente.
- Falhas de leitura ou gravação do armazenamento não impedem o uso dos dados da rede.

5. Testes e verificação

Evidências executadas sobre a implementação final

Verificação	Resultado
Jest	38 testes aprovados em 6 suítes
TypeScript	typecheck sem erros
Build Expo Web	exportação concluída e publicada no GitHub Pages
Integração ao vivo	login real com emilys e 12 produtos retornados pela DummyJSON
Playwright	build web de produção com a API real: login, sessão, catálogo e cache; modo offline simulado bloqueando a requisição do catálogo
GitHub	repositório tamyresogasawara/reuse-mobile-fiap-sprint-2

Cenários automatizados

- Autenticação externa e normalização da resposta.
- Persistência, restauração, expiração e encerramento da sessão.
- Validação do formulário e credenciais recusadas pela API.
- Catálogo obtido pela API e salvo no cache; cache recente sem chamada de rede.
- Fallback do cache expirado; falhas de leitura e gravação do cache.
- Erro sem cache com retry; navegação autenticada, catálogo online e logout.
- Fluxos anteriores preservados: busca, favoritos, anúncios, perfil e gamificação.

6. Conclusão

Resultado e aderência ao objetivo da atividade

A ReUse atende aos três eixos de avaliação. A API sustenta tanto o acesso quanto o catálogo online. A sessão persistente protege o fluxo principal e melhora a continuidade de uso. O cache local reduz chamadas desnecessárias e mantém o catálogo disponível durante falhas temporárias de conexão.

A integração foi feita sobre a base da Sprint 2, preservando a identidade visual, as animações, a gamificação e os fluxos locais, com separação entre telas, serviços, estado e testes.

Links finais

Código-fonte: https://github.com/tamyresogasawara/reuse-mobile-fiap-sprint-2

Aplicação web: https://tamyresogasawara.github.io/reuse-mobile-fiap-sprint-2/

ENTREGA INDIVIDUAL · TAMYRES OGASAWARA MACEDO